

Real-Time Payment Fraud Detection

Data Sources, Synthetic Generation, and Cleaning Pipeline

Agenda

- 1 Overview
- 2 Base Dataset
- 3 Synthetic Data Generation
- 4 Leakage Validation
- 5 Data Cleaning
- 6 Summary

Project Overview

- Data foundation for a fraud-detection pipeline, built in two stages:
 - ① **Synthetic Data Generation** — add 12 behavioral/contextual fields to the base transactional dataset
 - ② **Data Cleaning** — validate and flag data-quality issues without destroying fraud signal
- Base dataset: **PaySim** (Kaggle), **6,362,620** transactions
- Both stages processed the **complete dataset** — no sampling, no shortcuts

Base Dataset (11 cols) → Synthetic Generation (+12) → Cleaning (+3)
→ 26 cols

Development Process

Survey real data → Design spec → Implementation plan (TDD) →
Code task-by-task (RED → GREEN → commit) → Independent review →
Run on full 6.36M rows → Whole-branch review

- Every design decision is based on **measured** properties of the real data, not assumptions
- Every function is **test-driven**: test written first, then implementation
- Every task is **independently reviewed** for spec compliance and code quality
- **70 automated tests**, all passing

Base Dataset: PaySim

- *Online Payments Fraud Detection Dataset* (Kaggle), from the PaySim agent-based mobile-money simulator (Lopez-Rojas et al., 2016)
- Simulates customers and merchants transacting over one month
- A subset of transactions is labeled fraudulent — **account-takeover** behavior

Records	6,362,620
Columns	11 (step, type, amount, balances, identifiers, isFraud, isFlaggedFraud)
Fraud rate	8,213 / 6,362,620 = 0.1291%

Empirical Profiling — Three Findings That Shaped the Design

① Fraud is confined to two transaction types.

TRANSFER (4,097/532,909) and CASH_OUT (4,116/2,237,500) only.

PAYMENT, CASH_IN, DEBIT → zero fraud.

⇒ fraud here = **account-takeover**, not checkout fraud.

② Customer history is negligible.

99.85% of nameOrig values are unique (appear once).

⇒ fields generated at **row level**, not via a customer profile.

③ step allows exact time derivation.

Elapsed hours, no gaps, range 1–743.

⇒ time-of-day features are **derived**, not sampled.

Why Synthetic Data?

The base dataset has only raw financial fields. It lacks:

- Device fingerprinting
- IP-to-billing geolocation consistency
- Account tenure
- Payment-retry behavior
- Temporal usage patterns

⇒ **12 contextual fields** generated in Python (Faker + custom business logic).

- Fully vectorized numpy/pandas — no row-wise loops on 6.36M rows
- Single seeded RNG (seed=42) — exact reproducibility, verified
- 3 focused modules: geo-distance utility, field generation, leakage validation
- Developed **test-first**: **52 unit tests**, all passing

Five Design Principles

- ① **Row-level generation** — no persistent customer profile (justified by data)
- ② **Selective conditioning** — only behaviorally plausible fields depend on `isFraud`
- ③ **Bounded effect sizes** — conditional parameters capped at $2\text{--}4\times$ baseline
- ④ **Deterministic derivation** where possible — formula over randomness
- ⑤ **Post-hoc, objective leakage validation** — measured, not judged

The 12 Synthetic Fields (1/2)

Field	Mechanism	Base → Fraud
hour_of_day	Derived	(step - 1) mod 24
is_night_transaction	Derived	hour $\in [0,5]$
customer_account_age_days	Log-normal	median 400 → 275 days
device_id	Independent	pool of 50,000 UUIDs
browser	Independent	Chrome 55% / Safari 20% / ...
device_type	Independent	mobile 65% / desktop 30% / tablet 5%

The 12 Synthetic Fields (2/2)

Field	Mechanism	Base → Fraud
new_device_flag	Bernoulli	4% → 12%
billing_country	Independent	20 fixed countries
ip_country	Bernoulli	match rate 93% → 80%
ip_billing_distance_km	Derived	haversine(ip, billing)
shipping_billing_mismatch	Bernoulli	5% → 15%
failed_payment_attempts_24h	Poisson	mean 0.15 → 0.6

Full formulas, ranges, and business assumptions: Data Dictionary (auto-generated).

A field that correlates too strongly with `isFraud` trivializes the model. Each field is checked **objectively** after generation:

- **Continuous / Boolean fields:** univariate AUC, $\max(\text{AUC}, 1 - \text{AUC})$ — direction-agnostic
- **Categorical fields:** Cramér's V with Bergsma's (2013) small-sample bias correction

Fail thresholds (fixed in advance): $\text{AUC} \geq 0.75$ or corrected Cramér's $V \geq 0.50$

Issue #1 Found and Fixed: Account Age

Before

- Fraud median: 150 days
- Baseline median: 400 days
- AUC = **0.8753**
- **FAIL** (threshold 0.75)

After

- Fraud median narrowed to **275 days**
- Signal reduced, not removed
- AUC = **0.6689**
- **PASS**

Found on the **first real run** against all 6.36M rows — not a subsample.

Issue #2 Found and Fixed: Cramér's V Bias

- `device_id` has 50,000 distinct values — generated **independently** of fraud by construction
- Naive Cramér's V formula is upward-biased on sparse contingency tables
- On a smaller data subset, the naive formula reported ≈ 0.48 — dangerously close to the 0.50 threshold, despite zero true correlation

Fix: substituted Bergsma's (2013) bias-corrected estimator everywhere.

`device_id` corrected association: $0.0879 \rightarrow 0.0000$

Found via **independent review of the validation code**, not just its output.

Final Leakage Validation Results

Field	Metric	Value	Status
hour_of_day	AUC	0.6336	PASS
is_night_transaction	AUC	0.6217	PASS
customer_account_age_days	AUC	0.6689	PASS
device_id	Cramér's V	0.0000	PASS
browser	Cramér's V	0.0000	PASS
device_type	Cramér's V	0.0004	PASS
new_device_flag	AUC	0.5419	PASS
billing_country	Cramér's V	0.0000	PASS
ip_country	Cramér's V	0.0049	PASS
ip_billing_distance_km	AUC	0.5651	PASS
shipping_billing_mismatch	AUC	0.5495	PASS
failed_payment_attempts_24h	AUC	0.6589	PASS

12 / 12 fields PASS — on the full 6,362,620-row dataset

- Every field documented: name, type, unit, range, mechanism, formula, business assumption, measured association
- **Generated programmatically** by the same module that runs the leakage checks — cannot drift from the code
- Published in Markdown (satisfies Markdown/Excel deliverable requirement)

Field	Generation	Assumption
<code>new_device_flag</code>	Bernoulli, $p=0.04 \rightarrow 0.12$	Unrecognized device $\Rightarrow$ more likely fraud, not certain
<code>failed_payment_attempts_24h</code>	Poisson, $\lambda=0.15 \rightarrow 0.6$	Attackers retry before success

A row is removed only when it is structurally corrupt.

Anything that could be fraud signal is flagged, never removed.

Rare fraud cases often look statistically “unusual.” An indiscriminate cleaning rule risks deleting exactly the signal the model needs to learn.

Structural Checks — All Clear

Check	Result
Missing values (critical columns)	0 removed
Duplicates (full-row)	0 removed
Invalid categorical / Boolean values	0 removed

Plus a broader preliminary survey (negative balances, identifier format, step gaps, cross-field invariants) — **all zero violations**.

The dataset is structurally clean. These checks remain as a safeguard for future, less-curated input.

Outlier Treatment — Real Findings, All Flagged

- **Amount outliers** (Tukey $1.5 \times \text{IQR}$ fence): **338,078** transactions (5.31%)
Large amounts can themselves be fraud signal — kept, flagged.
- **Zero-amount transactions**: **16** transactions, all CASH_OUT
All 16 are confirmed fraud — removing them would delete real fraud examples.
- **Balance inconsistency**: **5,118,892** transactions (80.45%)
A known PaySim characteristic (merchant balances untracked) — **not a data error**.

Check	Rows before	Rows affected	Action
Missing values	6,362,620	0	Removed
Duplicates	6,362,620	0	Removed
Invalid values	6,362,620	0	Removed
Amount outliers	6,362,620	338,078 (5.31%)	Flagged, retained
Zero-amount	6,362,620	16	Flagged, retained
Balance inconsistency	6,362,620	5,118,892 (80.45%)	Flagged, retained

23 columns → **26 columns**. Row count and fraud rate (0.1291%) unchanged.

Implementation & Testing

- **70 automated unit tests**, all passing (52 generation + 18 cleaning)
- Test-driven development throughout: test first, then code
- 100% vectorized — no per-row loops over 6.36M records
- Single seeded RNG — fully reproducible runs
- Every task independently reviewed (spec compliance + code quality)
- Whole-branch review at the end of each stage — numbers re-verified independently, not just trusted

Limitations

- Conditional generation parameters are **explicit business assumptions**, not empirically calibrated — no public ground truth exists
- `shipping_billing_mismatch` reframed to an account-takeover interpretation, consistent with PaySim's actual fraud mechanism
- Fields generated at the **transaction level** (no customer profile) — by design, given the data
- The $1.5 \times \text{IQR}$ fence is a convention, not the only valid threshold
- Outlier/inconsistency flags mark rows; using them as model features is a modeling-stage decision

Key Takeaways

- Full 6,362,620-row dataset processed end-to-end — no sampling at any stage
- 12 synthetic fields, **12/12** pass objective leakage validation
- Cleaning preserves **100% of rows**; 3 fraud-relevant anomalies flagged, not discarded
- Validation caught **2 real issues** during development — proof the process works, not a formality
- Every artifact (data dictionary, cleaning report) is **auto-generated from code** — cannot drift from what was executed

Thank You

Questions?